

컴퓨터 비전

동양미래대학교
인공지능소프트웨어학과
백찬은 교수

3차원 비전

01

Point Cloud

02

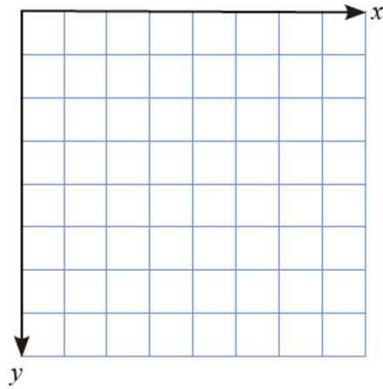
PointNet

03

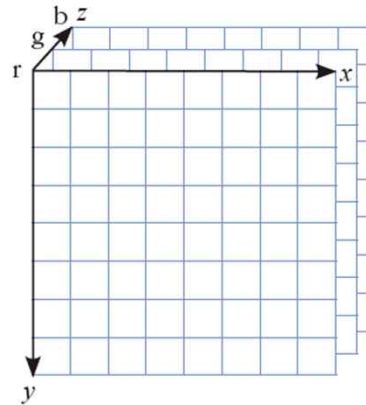
(실습) Point Cloud 영상의 분류



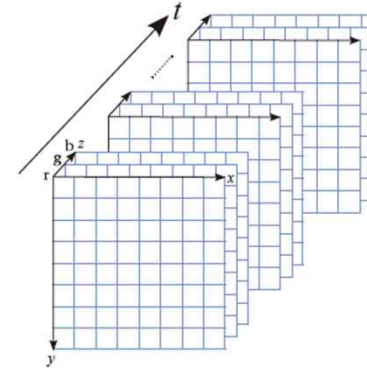
Point Cloud



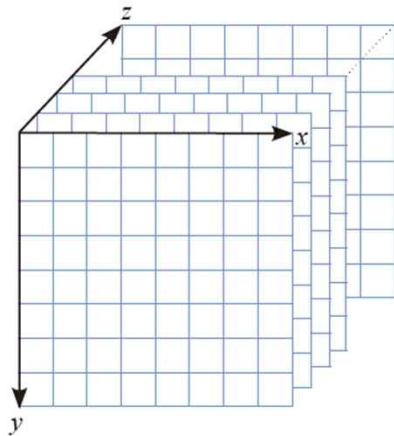
(a) 명암 영상



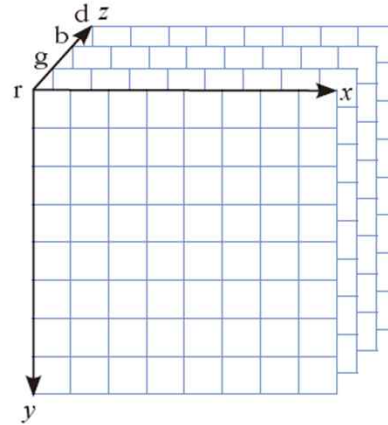
(b) 컬러 영상



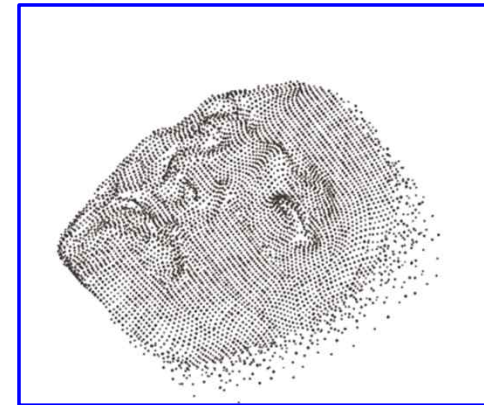
(c) 컬러 동영상



(d) 다분광/초분광/MR/CT 영상



(e) RGB-D 영상



(f) 점 구름 영상

그림 3-5 다양한 형태의 디지털 영상

Point Cloud

Point Cloud

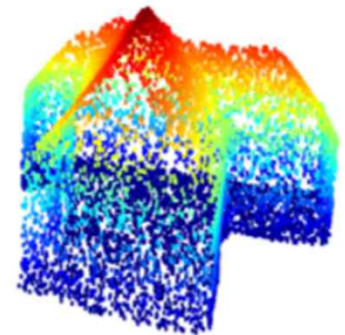
- 3차원 공간 안의 점들을 모아놓은 데이터 구조
- 실세계의 3D 형태를 수많은 작은 점들로 표현한 데이터
- **LiDAR(Light Detection And Ranging), Depth Camera, Stereo Camera** 등을 통해 형성됨
- **LiDAR**: 레이저를 발사하여 반사되어 들어오는 시간을 측정하여 물체와의 거리를 계산해 얻음
- **Stereo Camera(스테레오 비전)**: 카메라 두 대의 시차(disparity)를 이용해 깊이를 계산함



(a)



(b)



(c)

PointNet

- 3D Point Cloud를 입력으로 받아 처리하는 딥러닝 모델
- 용도: 3D 객체 분류, 3D 객체 분할, Scene Segmentation
- 기존 CNN은 격자 구조 내에 있는 이미지, voxel 등만 처리할 수 있었지만, 규칙적이지 않고 순서가 없는 점들의 집합인 point cloud를 처리하기 어려움
- 이런 문제를 해결하기 위해 나온 네트워크
 - 점의 순서와 상관없으며
 - 3D 공간의 회전과 변환에도 불변하며
 - 빠르고 효율적인 모델
- 3D 데이터를 격자로 변환하지 않고 그대로 처리한 모델 - N개의 점 (x, y, z) 좌표만 사용
- PointNet 내부에는 T-Net이라는 작은 네트워크 2개가 있음
- T-Net은 입력 점들의 회전/이동 보정 등의 역할을 하여 불변성을 확보함

PointNet

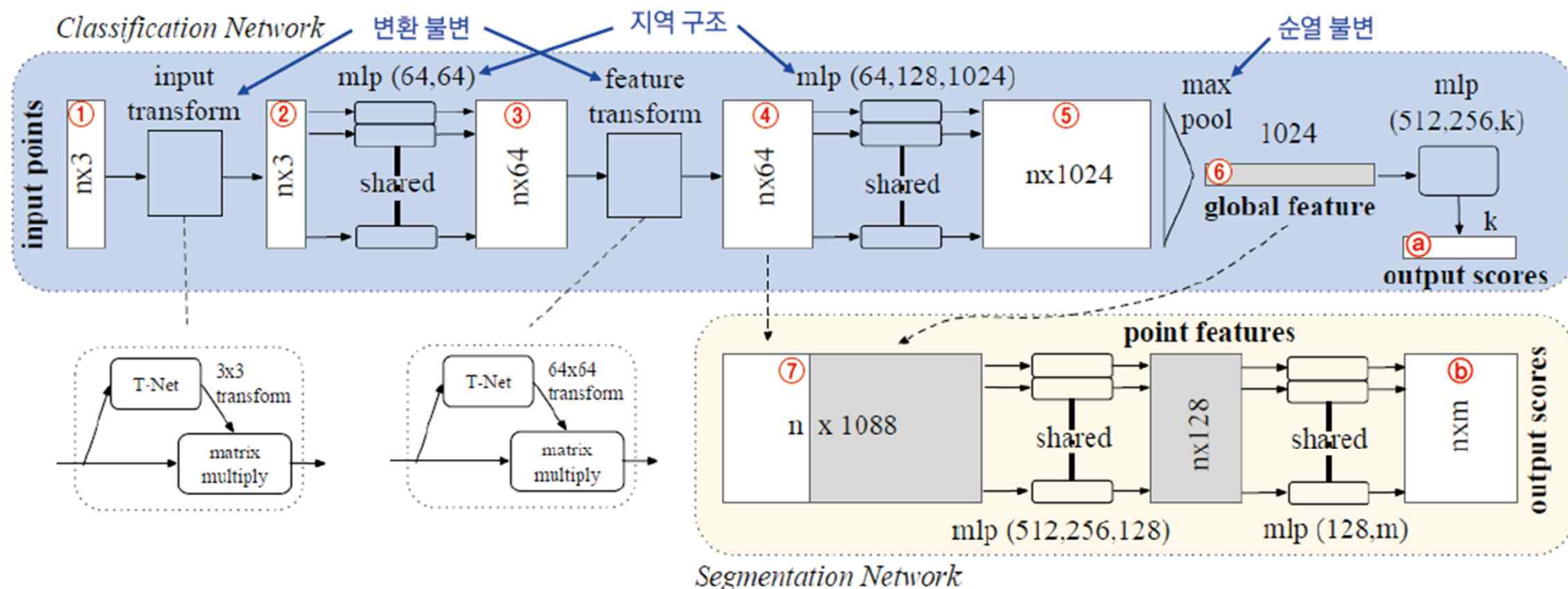
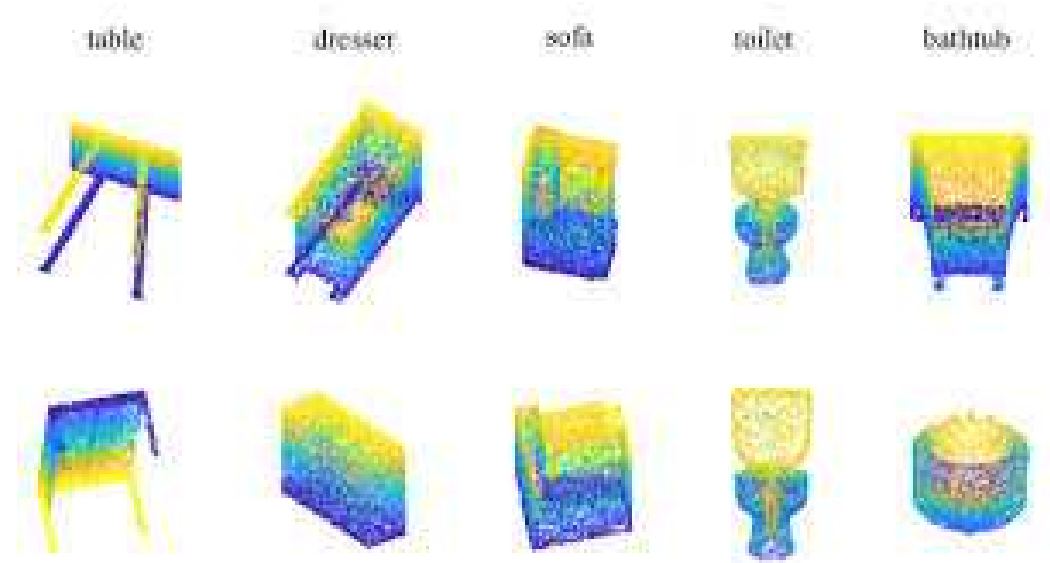


그림 12-18 PointNet의 구조와 동작[Qi2017]

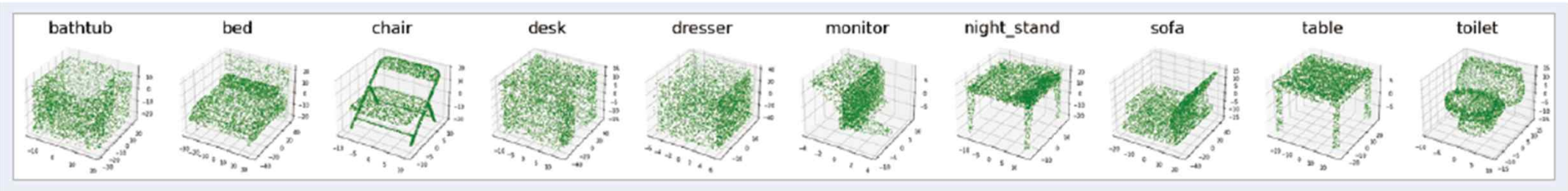
ModelNet 데이터셋

- Princeton에서 공개한 3D CAD 모델 데이터셋
- 3D 딥러닝 연구에서 널리 쓰이는 표준 벤치마크
- <https://modelnet.cs.princeton.edu/> 에서 다운받을 수 있음
- (예) ModelNet10: bed, chair, sofa, toilet, dresser, table, nightstand 등
- (예) ModelNet40: airplane, chair, sofa, lamp, stairs 등



<https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTf8WGv4s9kADx9zfEq41ikliJCAQltGk8TA&s>

(실습) ModelNet 데이터셋을 전처리하여 Point Cloud 데이터 생성하기



프로그램 12-1

ModelNet 데이터셋에서 점 구름을 생성하고 디스플레이하기

```
01 import os
02 import trimesh
03 import tensorflow as tf
04 import matplotlib.pyplot as plt
05
06 classes=['bathtub','bed','chair','desk','dresser','monitor','night_stand',
07         'sofa','table','toilet']
08
09 path='http://3dvision.princeton.edu/projects/2014/3DShapeNets/ModelNet10.zip'
10 data_dir=tf.keras.utils.get_file('modelnet.zip',path,extract=True)
11 data_dir=os.path.join(os.path.dirname(data_dir),'ModelNet10')
```


(실습) ModelNet 데이터셋을 전처리하여 Point Cloud 데이터 생성하기

```
11
12 fig=plt.figure(figsize=(50,5))
13 for i in range(len(classes)):
14     mesh=trimesh.load(os.path.join(data_dir,classes[i]+'train/'+classes[i]
15                                     +'_0001.off'))
16
17     points=mesh.sample(4096)
18
19     ax=fig.add_subplot(1,10,i+1,projection='3d')
20     ax.set_title(classes[i],fontsize=30)
21     ax.scatter(points[:,0],points[:,1],points[:,2],s=1,c='g')
```

오류 발생시 해당 폴더에 들어가 압축 풀기

```
ValueError: string is not a file: `C:\Users\abc\.keras\datasets\ModelNet10\bathtub/train/bathtub_0001.off`
<Figure size 7200x720 with 0 Axes>

In [3]:
```

(실습) PointNet을 이용하여 Point Cloud 영상 분류하기

프로그램 12-2

PointNet을 이용해 점 구름 영상 분류하기

```
01  import os
02  import glob
03  import trimesh
04  import numpy as np
05  import tensorflow as tf
06  from tensorflow import keras
07  from tensorflow.keras import layers
08  import matplotlib.pyplot as plt
09
10  classes=['bathtub','bed','chair','desk','dresser','monitor','night_stand',
11          'sofa','table','toilet']
12
13  path="http://3dvision.princeton.edu/projects/2014/3DShapeNets/ModelNet10.zip"
14  data_dir=tf.keras.utils.get_file('modelnet.zip',path,extract=True)
15  data_dir=os.path.join(os.path.dirname(data_dir),'ModelNet10')
```

(실습) PointNet을 이용하여 Point Cloud 영상 분류하기

```
16 def parse_dataset(num_points=2048):
17     train_points, train_labels=[],[]
18     test_points, test_labels=[],[]
19
20     for i in range(len(classes)):
21         folder=os.path.join(data_dir, classes[i])
22         print('데이터 읽기: 부류 {}'.format(os.path.basename(folder)))
23         train_files=glob.glob(os.path.join(folder, 'train/*'))
24         test_files=glob.glob(os.path.join(folder, 'test/*'))
25
26         for f in train_files:
27             train_points.append(trimesh.load(f).sample(num_points))
28             train_labels.append(i)
29         for f in test_files:
30             test_points.append(trimesh.load(f).sample(num_points))
31             test_labels.append(i)
32     return(np.array(train_points), np.array(test_points), np.array(train_
33            labels), np.array(test_labels))
```

```

34 NUM_POINTS=2048          # 샘플 영상의 점의 개수
35 NUM_CLASSES=10           # 부류 개수
36 batch_size=32
37
38 x_train,x_test,y_train,y_test=parse_dataset(NUM_POINTS)
39
40 def conv_bn(x,filters):    # 컨볼루션층과 배치 정규화
41     x=layers.Conv1D(filters,kernel_size=1,padding='valid')(x)
42     x=layers.BatchNormalization(momentum=0.0)(x)
43     return layers.Activation('relu')(x)
44
45 def dense_bn(x,filters):   # 완전연결층과 배치 정규화
46     x=layers.Dense(filters)(x)
47     x=layers.BatchNormalization(momentum=0.0)(x)
48     return layers.Activation('relu')(x)
49
50 class OrthogonalRegularizer(keras.regularizers.Regularizer):
51     def __init__(self,num_features,l2reg=0.001):
52         self.num_features=num_features
53         self.l2reg=l2reg
54         self.eye=tf.eye(num_features)
55
56     def __call__(self,x):
57         x=tf.reshape(x,(-1,self.num_features,self.num_features))
58         xxt=tf.tensordot(x,x,axes=(2,2))
59         xxt=tf.reshape(xxt,(-1,self.num_features,self.num_features))
60         return tf.reduce_sum(self.l2reg*tf.square(xxt-self.eye))

```

```

61
62 def tnet(inputs,num_features): # T-Net
63     bias=keras.initializers.Constant(np.eye(num_features).flatten())
64     reg=OrthogonalRegularizer(num_features)
65
66     x=conv_bn(inputs,32)
67     x=conv_bn(x,64)
68     x=conv_bn(x,512)
69     x=layers.GlobalMaxPooling1D()(x)
70     x=dense_bn(x,256)
71     x=dense_bn(x,128)
72     x=layers.Dense(num_features*num_features,kernel_initializer='zeros',bias_
        initializer=bias,activity_regularizer=reg)(x)
73
74     feat_T=layers.Reshape((num_features, num_features))(x)
75     return layers.Dot(axes=(2,1))([inputs,feat_T])
        # 특징 맵에 어파인 변환(3*3행렬) 적용
76

```


학습 시간이 많이 걸리므로, epoch=2로 수정하여 코딩하기

```
77 inputs=keras.Input(shape=(NUM_POINTS,3)) # PointNet의 입력
78 x=tnet(inputs,3) # PointNet 구축 (특징 맵은 논문의 반절로 설정)
79 x=conv_bn(x,32)
80 x=conv_bn(x,32)
81 x=tnet(x,32)
82 x=conv_bn(x,32)
83 x=conv_bn(x,64)
84 x=conv_bn(x,512)
85 x=layers.GlobalMaxPooling1D()(x)
86 x=dense_bn(x,256)
87 x=layers.Dropout(0.3)(x)
88 x=dense_bn(x,128)
89 x=layers.Dropout(0.3)(x)
90 outputs=layers.Dense(NUM_CLASSES,activation='softmax')(x) # PointNet 출력
91
92 model=keras.Model(inputs=inputs,outputs=outputs,name='pointnet')
93
94 model.compile(loss='sparse_categorical_crossentropy',optimizer=keras.
    optimizers.Adam(learning_rate=0.001),metrics=["sparse_categorical_accuracy"])
95 model.fit(x_train,y_train,epochs=20,validation_data=(x_test,y_test))
```

(실습) PointNet을 이용하여 Point Cloud 영상 분류하기

```
96
97 chosen=np.random.randint(0,len(x_test),8)
98 points=x_test[chosen]          # 앞에 배치된 8개 샘플을 예측해봄
99 labels=y_test[chosen]
100
101 preds=model.predict(points)
102 preds=tf.math.argmax(preds,-1)
103
104 fig=plt.figure(figsize=(15,4))  # 예측 결과를 시각화
105 for i in range(8):
106     ax=fig.add_subplot(2,4,i+1,projection='3d')
107     ax.scatter(points[i,:,0],points[i,:,1],points[i,:,2],s=1,c='g')
108     ax.set_title('pred:{:}, GT:{:}'.format(classes[preds[i].numpy()],classes
109               [labels[i]]),fontsize=16)
109     ax.set_axis_off()
```


데이터 읽기: 부류 bathtub

데이터 읽기: 부류 bed

...

데이터 읽기: 부류 toilet

Epoch 1/20

125/125 [=====] - 22s 150ms/step - loss: 3.5033 - sparse_categorical_accuracy: 0.2886 - val_loss: 383046.3125 - val_sparse_categorical_accuracy: 0.2258

Epoch 2/20

125/125 [=====] - 18s 145ms/step - loss: 2.8524 - sparse_categorical_accuracy: 0.4475 - val_loss: 698092000714608869376.0000 - val_sparse_categorical_accuracy: 0.4361

...

Epoch 20/20

125/125 [=====] - 18s 146ms/step - loss: 1.5803 - sparse_categorical_accuracy: 0.8464 - val_loss: 22705.3477 - val_sparse_categorical_accuracy: 0.8150

pred:sofa, GT:sofa



pred:bathtub, GT:bathtub



pred:desk, GT:desk



pred:bed, GT:bed



pred:table, GT:table



pred:monitor, GT:monitor



pred:sofa, GT:bathtub



pred:bed, GT:bed



생성 비전

01

생성 모델 기초

02

오토인코더

03

변분 오토인코더

04

GAN





생성 모델 기초

생성 모델

생성 모델

- 분류모델과 생성모델 예
 - 분류 모델: 진짜 데이터와 가짜 데이터를 분류
 - 생성 모델: 새로운 이미지를 만들어냄
- 생성 모델은 데이터의 분포를 배운 후, 그 분포에서 새로운 샘플을 만들어내는 것
- 대표적인 생성 모델들
 - Gaussian Mixture Model(GMM), 은닉 마르코프 모델(HMM), 오토인코더, 변분 오토인코더, GAN(Generative Adversarial Network), Diffusion Model 등

(실습) 키, 몸무게 쌍을 생성하는 모델 만들기

가우시안 기반 생성모델 적용

- 데이터가 가우시안 분포를 따른다고 가정하면, 평균과 공분산값만 있으면 새로운 샘플을 쉽게 생성 가능함
 - 공분산(covariance): 두 변수가 함께 어떻게 변하는지를 나타내는 값
 - (예) 키가 클수록 몸무게가 커지면 양(+)의 공분산
 - 시험 점수는 올라가는데, 스트레스는 줄어든다면 음(-)의 공분산
 - 서로 전혀 관계가 없으면 0에 가까운 공분산

(실습) 키, 몸무게 쌍을 생성하는 모델 만들기

프로그램 13-1

(키, 몸무게)를 생성하는 생성 모델 제작하기

```
01 import numpy as np
02
03 X=np.array([[169,70],[172,68],[175,78],[163,58],[180,80],[159,76],[158,52],[173,69],
04             [180,75],[155,50],[187,90],[170,66]])
05
06 m=np.mean(X,axis=0)                                # 모델 학습
07 cv=np.cov(X,rowvar=False)
08
09 gen=np.random.multivariate_normal(m,cv,5)           # 샘플 생성
10 print(gen)
```

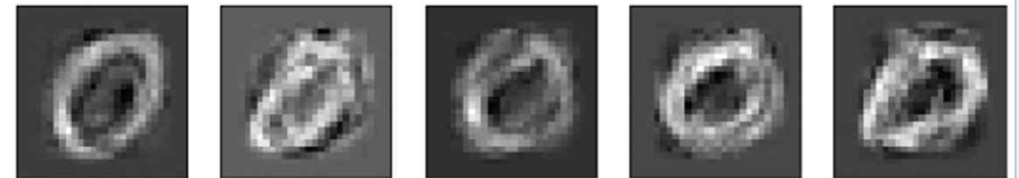
```
[[169.42576244 62.94325219], [161.14723842 61.16595917], [175.42762252
68.32468183], [170.42014907 62.96828809], [191.10965807 92.38479145]]
```


(실습) MNIST 숫자 데이터를 생성하는 가우시안 모델

프로그램 13-2

MNIST를 가우시안 모델링하고 샘플 생성하기

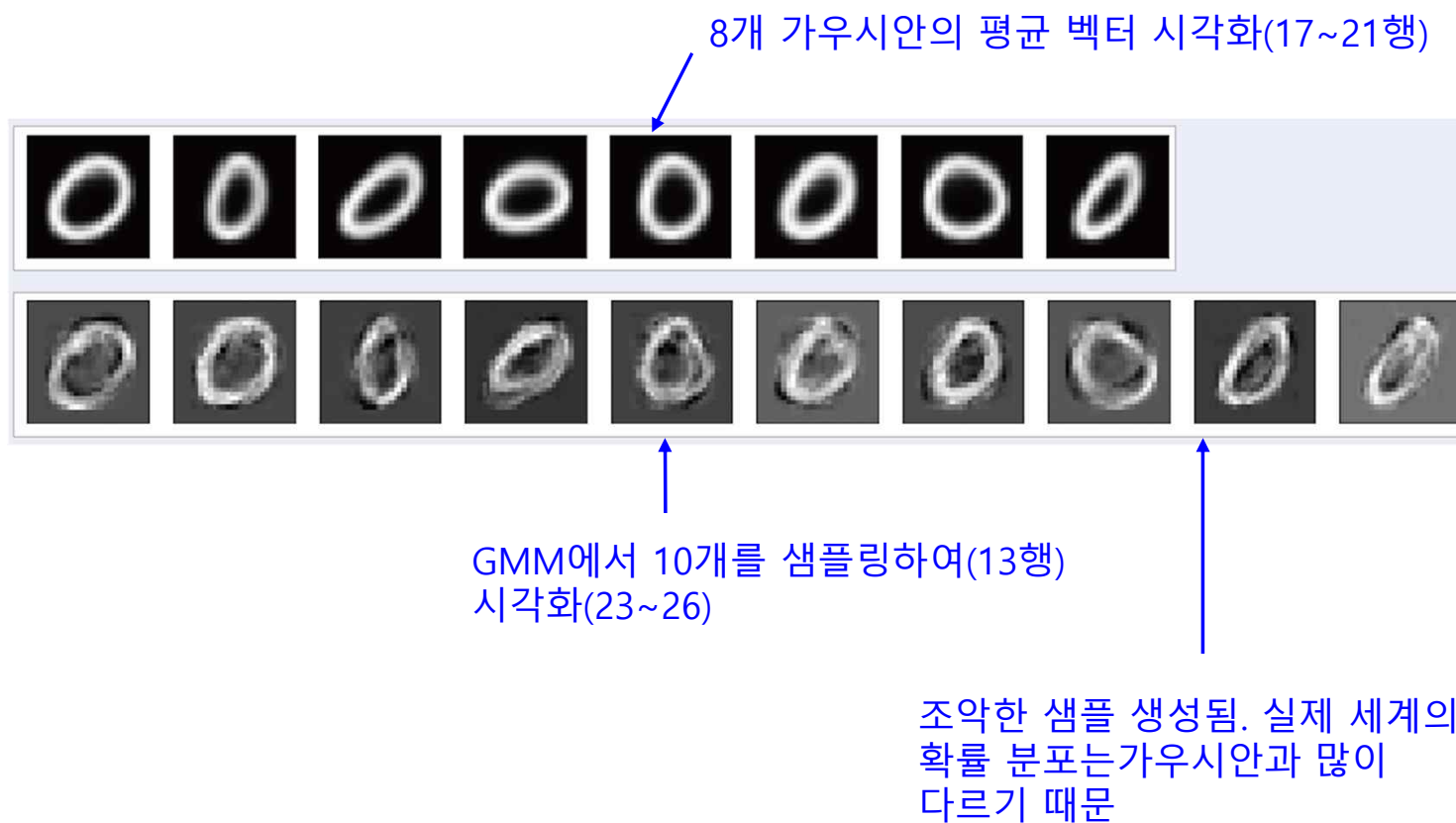
```
01 import numpy as np
02 from tensorflow.keras.datasets import mnist
03
04 (x_train,y_train),(x_test,y_test)=mnist.load_data() # 데이터셋 구성
05 X=x_train[np.isin(y_train,[0])]                  # 0부류만 추출
06 X=X.reshape((X.shape[0],28*28))
07
08 m=np.mean(X,axis=0)                             # 모델 학습
09 cv=np.cov(X,rowvar=False)
10
11 gen=np.random.multivariate_normal(m,cv,5)        # 샘플 생성
12
13 import matplotlib.pyplot as plt
14
15 plt.figure(figsize=(10,4))                       # 샘플 그리기
16 for i in range(5):
17     plt.subplot(1,5,i+1)
18     plt.imshow(gen[i].reshape((28,28)),cmap='gray'); plt.xticks([]); plt.
yticks([])
```



(실습) MNIST 숫자 데이터를 GMM으로 생성하기

가우시안 혼합 모델(GMM, Gaussian Mixture Model)

- 하나의 가우시안으로는 데이터가 여러 무리(클러스터)를 이루는 경우를 제대로 표현할 수 없음
- 가우시안 혼합 모델: 여러개의 가우시안 조합하여 복잡한 분포를 근사하는 모델



(실습) MNIST 숫자 데이터를 GMM으로 생성하기

프로그램 13-3

MNIST를 GMM으로 모델링하고 샘플 생성하기

```
01 import numpy as np
02 from tensorflow.keras.datasets import mnist
03 from sklearn.mixture import GaussianMixture
04
05 (x_train,y_train),(x_test,y_test)=mnist.load_data()
06 X=x_train[np.isin(y_train,[0])]
07 X=X.reshape((X.shape[0],28*28))
08
09 k=8 # 식 (13.4)의 가우시안 개수 k를 8로 설정
10
11 gm=GaussianMixture(n_components=k).fit(X)
12
13 gen=gm.sample(n_samples=10)
14
15 import matplotlib.pyplot as plt
16
17 plt.figure(figsize=(20,4)) # 학습된 가우시안 8개의 평균을 그림
18 for i in range(k):
19     plt.subplot(1,10,i+1)
20     plt.imshow(gm.means_[i].reshape((28,28)),cmap='gray'); plt.xticks([]); plt.
        yticks([])
21 plt.show()
22
23 plt.figure(figsize=(20,4)) # 생성된 샘플 10개를 그리기
24 for i in range(10):
25     plt.subplot(1,10,i+1)
26     plt.imshow(gen[0][i].reshape((28,28)),cmap='gray'); plt.xticks([]); plt.
        yticks([])
```



오토인코더

Autoencoder

Autoencoder(오토인코더)

- 입력된 데이터에서 주요 특징을 뽑아 압축하였다(latent vector) 다시 원래대로 복원하는 신경망
- 입력 -> 주요 특징들(latent vector) -> 다시 복원
- 데이터들에서 주요 특징들을 뽑아, latent vector가 있는 공간(latent space) 근처에서 적당한 점을 골라 다시 복원하는 모델에 입력하면 새로운 이미지가 생성됨
- 즉, 이 구조를 생성 모델처럼 활용 가능함
- 단점:
 - 단순한 오토인코더 구조에서는 latent space에서 적당한 점을 골라 decoder에 입력하면 엉뚱한 이미지가 생성됨
 - 해결책: 변분 오토인코더

Autoencoder

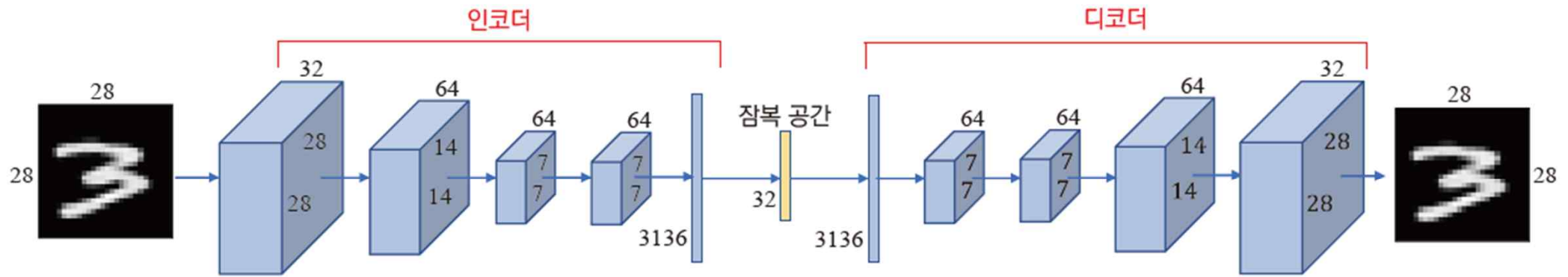


그림 13-6 생성 모델로서 오토인코더

(실습) MNIST 숫자 데이터를 오토인코더로 생성하기

프로그램 13-4

MNIST를 오토인코더로 모델링하고 샘플 생성하기

```
01 import numpy as np
02 from tensorflow.keras.datasets import mnist
03 from tensorflow.keras.layers import Input,Dense,Flatten,Reshape,Conv2D,Conv2
   DTranspose
04 from tensorflow.keras.models import Model
05
06 (x_train,y_train),(x_test,y_test)=mnist.load_data()
                                # MNIST를 읽고 신경망에 입력할 준비
07 x_train=x_train.astype('float32')/255.
08 x_test=x_test.astype('float32')/255.
09 x_train=np.reshape(x_train,(len(x_train),28,28,1))
10 x_test=np.reshape(x_test,(len(x_test),28,28,1))
11
12 zdim=32                      # 잠재 공간의 차원
```


(실습) MNIST 숫자 데이터를 오토인코더로 생성하기

```
13
14 encoder_input=Input(shape=(28,28,1))          # 인코더
15 x=Conv2D(32,(3,3),activation='relu',padding='same',strides=(1,1))(encoder_
    input)
16 x=Conv2D(64,(3,3),activation='relu',padding='same',strides=(2,2))(x)
17 x=Conv2D(64,(3,3),activation='relu',padding='same',strides=(2,2))(x)
18 x=Conv2D(64,(3,3),activation='relu',padding='same',strides=(1,1))(x)
19 x=Flatten()(x)
20 encoder_output=Dense(zdim)(x)
21 model_encoder=Model(encoder_input,encoder_output)
22
23 decoder_input=Input(shape=(zdim,))             # 디코더
24 x=Dense(3136)(decoder_input)
25 x=Reshape((7,7,64))(x)
26 x=Conv2DTranspose(64,(3,3),activation='relu',padding='same',strides=(1,1))(x)
27 x=Conv2DTranspose(64,(3,3),activation='relu',padding='same',strides=(2,2))(x)
28 x=Conv2DTranspose(32,(3,3),activation='relu',padding='same',strides=(2,2))(x)
29 x=Conv2DTranspose(1,(3,3),activation='relu',padding='same',strides=(1,1))(x)
30 decoder_output=x
31 model_decoder=Model(decoder_input,decoder_output)
32
33 model_input=encoder_input                       # 인코더와 디코더를 결합하여 오토인코더 모델 구축
34 model_output=model_decoder(encoder_output)
35 model=Model(model_input,model_output)
```

(실습) MNIST 숫자 데이터를 오토인코더로 생성하기

```
36
37 model.compile(optimizer='Adam',loss='mse') # 오토인코더 학습
38 model.fit(x_train,x_train,epochs=50,batch_size=128,shuffle=True,validation_
    data=(x_test,x_test))
39
40 import matplotlib.pyplot as plt
41
42 i=np.random.randint(x_test.shape[0]) # 테스트 집합에서 임의로 두 샘플 선택
43 j=np.random.randint(x_test.shape[0])
44 x=np.array((x_test[i],x_test[j]))
45 z=model_encoder.predict(x)
46
47 zz=np.zeros((11,zdim)) # 두 샘플 사이에서 등간격 점을 만듦
48 alpha=[0.0,0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9,1.0]
49 for i in range(11):
50     zz[i]=(1.0-alpha[i])*z[0]+alpha[i]*z[1]
51
52 gen=model_decoder.predict(zz) # 등간격 점에서 가짜 샘플 생성
53
54 plt.figure(figsize=(20,4))
55 for i in range(11):
56     plt.subplot(1,11,i+1)
57     plt.imshow(gen[i].reshape(28,28),cmap='gray'); plt.xticks([]); plt.yticks([])
58     plt.title(str(alpha[i]))
```

(실습) MNIST 숫자 데이터를 오토인코더로 생성하기

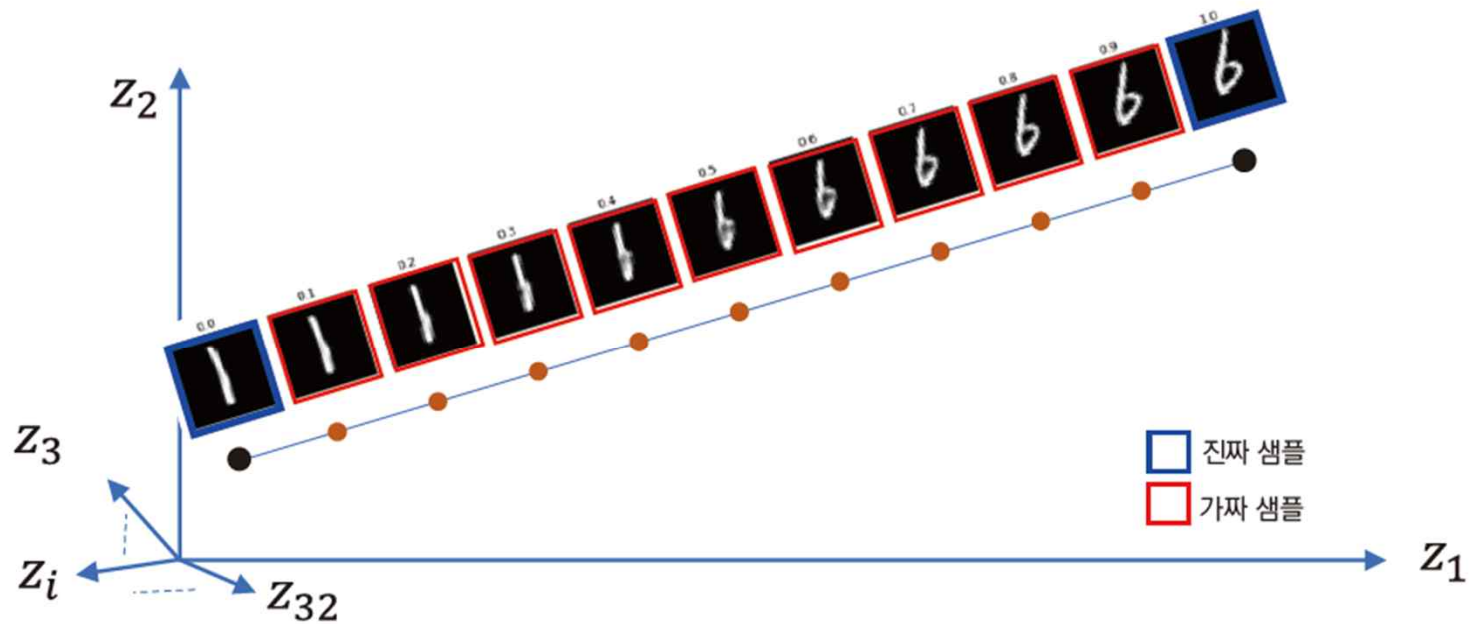


그림 13-7 두 개의 진짜 샘플 사이에서 가짜 샘플 생성



변분 오토인코더

Variational Autoencoder

Variational Autoencoder(변분 오토인코더, VAE)

- 기존 autoencoder의 latent space에서 latent vector 근처의 점을 다시 복원하면 이상한 이미지가 나오는 문제를 해결하기 위해 고안된 모델
- 여기에서는 하나의 고정된 latent vector가 아닌, **확률적 분포(평균 μ , 분산 σ)**를 출력하게 만들어 latent space가 연속적이며 매끄럽게 정렬된 공간으로 나옴
- 즉, 잘 정렬된 latent space를 만들도록 학습하며, 아무 점을 선택하여 복원하여도 자연스러운 이미지가 생성되도록 개선하였음
- 장점: 생성되는 이미지가 안정적임
- 단점: VAE로 생성한 이미지가 GAN으로 생성한 이미지보다 흐린 경향이 있음

(실습) MNIST 숫자 데이터를 VAE로 생성하기

```
1  import numpy as np
2  import tensorflow as tf
3  import keras
4  from keras import layers
5  import keras.ops as ops
6  import matplotlib.pyplot as plt
7
8  # -----
9  # 1. MNIST 데이터 불러오기 및 전처리
10 # -----
11 (x_train, _), (x_test, _) = keras.datasets.mnist.load_data()
12
13 # 실수형 변환 및 정규화
14 x_train = x_train.astype("float32") / 255.
15 x_test = x_test.astype("float32") / 255.
16
17 # CNN 입력 형태(28x28x1)로 변환
18 x_train = np.reshape(x_train, (-1, 28, 28, 1))
19 x_test = np.reshape(x_test, (-1, 28, 28, 1))
20
21
```

(실습) MNIST 숫자 데이터를 VAE로 생성하기

```
21
22 # -----
23 # 2. Sampling Layer 구현
24 #   VAE에서  $z = \mu + \sigma * \epsilon$  를 계산하는 부분
25 # -----
26 class Sampling(layers.Layer):
27     def call(self, inputs):
28         z_mean, z_log_var = inputs
29
30         #  $\epsilon \sim N(0, 1)$ 
31         epsilon = tf.random.normal(shape=tf.shape(z_mean))
32
33         #  $z = \text{mean} + \exp(\log\_var / 2) * \epsilon$ 
34         return z_mean + tf.exp(0.5 * z_log_var) * epsilon
35
36
37 # -----
38 # 3. Encoder 모델
39 #   입력 이미지 → 잠복공간(latent vector) z로 압축
40 # -----
41 latent_dim = 32 # 잠복 공간 차원
42
43 encoder_inputs = keras.Input(shape=(28, 28, 1))
44
45 # CNN 기반 특징 추출
```


(실습) MNIST 숫자 데이터를 VAE로 생성하기

```
45 # CNN 기반 특징 추출
46 x = layers.Conv2D(32, 3, activation="relu", padding="same")(encoder_inputs)
47 x = layers.Conv2D(64, 3, activation="relu", padding="same", strides=2)(x)
48 x = layers.Conv2D(64, 3, activation="relu", padding="same", strides=2)(x)
49 x = layers.Flatten()(x)
50 x = layers.Dense(128, activation="relu")(x)
51
52 # 잠복 공간의 평균과 분산 계산
53 z_mean = layers.Dense(latent_dim)(x)
54 z_log_var = layers.Dense(latent_dim)(x)
55
56 # 재매개변수화(reparameterization trick)
57 z = Sampling()([z_mean, z_log_var])
58
59 # Encoder 모델 정의
60 encoder = keras.Model(encoder_inputs, [z_mean, z_log_var, z], name="encoder")
61
62
63 # -----
64 # 4. Decoder 모델
65 #   잠복공간 z → 이미지 복원
66 # -----
67 latent_inputs = keras.Input(shape=(latent_dim,))
68
69 x = layers.Dense(7 * 7 * 64, activation="relu")(latent_inputs)
```


(실습) MNIST 숫자 데이터를 VAE로 생성하기

```
69 x = layers.Dense(7 * 7 * 64, activation="relu")(latent_inputs)
70 x = layers.Reshape((7, 7, 64))(x)
71 x = layers.Conv2DTranspose(64, 3, activation="relu", padding="same", strides=2)(x)
72 x = layers.Conv2DTranspose(64, 3, activation="relu", padding="same", strides=2)(x)
73
74 # 출력은 원본 이미지와 동일한 28x28x1
75 decoder_outputs = layers.Conv2DTranspose(1, 3, activation="sigmoid", padding="same")(x)
76
77 # Decoder 모델 정의
78 decoder = keras.Model(latent_inputs, decoder_outputs, name="decoder")
79
80
81 # -----
82 # 5. VAE 모델 정의 (Custom training loop 사용)
83 #   손실 = 재구성 손실 + KL 발산
84 # -----
85 class VAE(keras.Model):
86     def __init__(self, encoder, decoder, **kwargs):
87         super().__init__(**kwargs)
88         self.encoder = encoder
89         self.decoder = decoder
90
91     # 학습 단계 정의
92     def train_step(self, x):
93         with tf.GradientTape() as tape:
94             z_mean, z_log_var, z = self.encoder(x)
95             reconstruction = self.decoder(z)
96
97             # 재구성 손실 (Binary Cross Entropy)
98             reconstruction_loss = ops.mean(
```

(실습) MNIST 숫자 데이터를 VAE로 생성하기

```

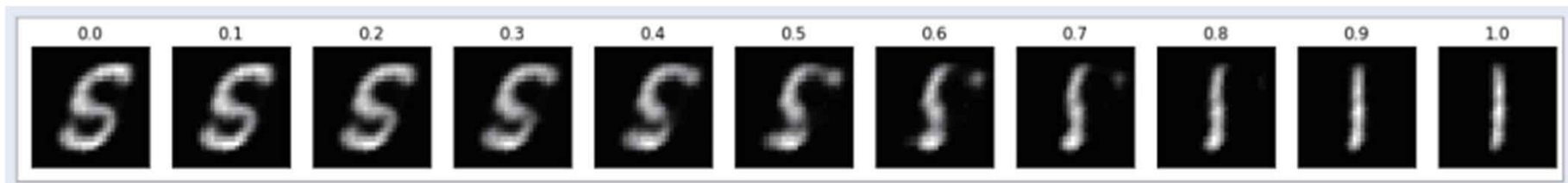
98         reconstruction_loss = ops.mean(
99             ops.sum(
100                 keras.losses.binary_crossentropy(x, reconstruction),
101                 axis=(1, 2)
102             )
103         )
104
105         # KL 발산
106         kl_loss = -0.5 * ops.mean(
107             ops.sum(
108                 1 + z_log_var - ops.square(z_mean) - ops.exp(z_log_var),
109                 axis=1
110             )
111         )
112
113         # 전체 손실
114         loss = reconstruction_loss + kl_loss
115
116         # 가중치 업데이트
117         grads = tape.gradient(loss, self.trainable_weights)
118         self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
119
120         return {"loss": loss}
121
122
123 vae = VAE(encoder, decoder)
124 vae.compile(optimizer=keras.optimizers.Adam())
125
```

(실습) MNIST 숫자 데이터를 VAE로 생성하기

```
127 # -----
128 # 6. 모델 학습
129 # -----
130 vae.fit(x_train, epochs=20, batch_size=128)
131
132
133 # -----
134 # 7. 잠복 공간 보간(latent interpolation)
135 #   두 이미지의 z 벡터를 선형보간하여 생성물 비교
136 # -----
137 i = np.random.randint(x_test.shape[0])
138 j = np.random.randint(x_test.shape[0])
139
140 # 두 개 테스트 이미지 선택
141 x = np.array((x_test[i], x_test[j]))
142
143 # 두 이미지의 z 벡터 추출
144 z_values = encoder.predict(x)[2]
145 z1, z2 = z_values[0], z_values[1]
146
147 # 0~1 사이를 11단계로 나누어 보간
148 alpha = np.linspace(0, 1, 11)
149 zz = np.array([(1 - a) * z1 + a * z2 for a in alpha])
150
151 # 보간된 z를 이용하여 이미지 생성
152 gen = decoder.predict(zz)
153
```

(실습) MNIST 숫자 데이터를 VAE로 생성하기

```
155 # -----  
156 # 8. 보간 결과 시각화  
157 # -----  
158 plt.figure(figsize=(20, 4))  
159 for i in range(11):  
160     plt.subplot(1, 11, i + 1)  
161     plt.imshow(gen[i].reshape(28, 28), cmap='gray')  
162     plt.xticks([])  
163     plt.yticks([])  
164     plt.title(f"{alpha[i]:.1f}")  
165 plt.show()  
166
```





GAN

GAN

Generative Adversarial Network(GAN, 생성형 적대 신경망)

- 생성자와 판별자가 경쟁하며 점점 정교한 이미지를 만들도록 학습되는 신경망
- 생성자(G): 위조지폐범 -> 가짜 데이터를 만듦
- 판별자(D): 경찰 -> 가짜 데이터를 적발함
- 생성자와 판별자를 경쟁하게 만들면서 둘 다 실력이 점점 향상되도록 만듦
- 장점: 선명하고, 사실적인 이미지를 생성할 수 있음

(실습) MNIST 숫자 데이터로 GAN으로 생성하기

E1: loss D(real)=0.328, D(fake)0.312 GAN 3.109
E2: loss D(real)=0.707, D(fake)0.618 GAN 1.142
E3: loss D(real)=0.382, D(fake)0.150 GAN 2.392
E4: loss D(real)=0.376, D(fake)0.338 GAN 1.957
...
E198: loss D(real)=0.603, D(fake)0.548 GAN 1.064
E199: loss D(real)=0.540, D(fake)0.591 GAN 1.115
E200: loss D(real)=0.637, D(fake)0.658 GAN 1.106



(실습) MNIST 숫자 데이터를 GAN으로 생성하기

```
1  import numpy as np
2  import tensorflow as tf
3  from tensorflow.keras.datasets import fashion_mnist
4  from tensorflow.keras.optimizers import Adam
5  from tensorflow.keras.models import Sequential
6  from tensorflow.keras.layers import Dense, Reshape, Flatten, Conv2D, Conv2DTranspose
7  from tensorflow.keras.layers import LeakyReLU, Dropout
8  import matplotlib.pyplot as plt
9
10 # -----
11 # 1. Fashion-MNIST 데이터 불러오기 및 전처리
12 # -----
13 (x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
14
15 # 클래스 8(Bag)만 선택
16 x_train = x_train[np.isin(y_train, [8])]
17
18 # [-1, 1] 구간으로 정규화 (tanh 출력과 맞추기 위함)
19 x_train = (x_train.astype("float32") / 255.0) * 2.0 - 1.0
20
21 # GAN 입력 형태를 위해 채널 차원 추가
22 x_train = np.expand_dims(x_train, axis=-1)
23
24 # 잠재 공간 차원 정의
25 zdim = 100
26
```

(실습) MNIST 숫자 데이터를 GAN으로 생성하기

```
27
28 # -----
29 # 2. 분별망(Discriminator) 정의
30 # -----
31 def make_discriminator(input_shape):
32     # 진짜와 가짜 이미지를 구분하는 이진 분류기
33     model = Sequential()
34
35     model.add(Conv2D(64, (3, 3), padding="same",
36                     input_shape=input_shape))
37     model.add(LeakyReLU(0.2))
38
39     model.add(Conv2D(128, (3, 3), strides=(2, 2), padding="same"))
40     model.add(LeakyReLU(0.2))
41
42     model.add(Conv2D(128, (3, 3), strides=(2, 2), padding="same"))
43     model.add(LeakyReLU(0.2))
44
45     model.add(Conv2D(256, (3, 3), strides=(2, 2), padding="same"))
46     model.add(LeakyReLU(0.2))
47
48     model.add(Flatten())
49     model.add(Dropout(0.4))
50
51     model.add(Dense(1, activation="sigmoid"))
52
53     model.compile(
```

(실습) MNIST 숫자 데이터를 GAN으로 생성하기

```
53     model.compile(  
54         loss="binary_crossentropy",  
55         optimizer=Adam(learning_rate=0.0002, beta_1=0.5),  
56         metrics=["accuracy"]  
57     )  
58  
59     return model  
60  
61  
62     # -----  
63     # 3. 생성망(Generator) 정의  
64     # -----  
65     def make_generator(zdim):  
66         # 잠복 벡터 z를 입력받아 28x28x1 이미지를 생성하는 모델  
67         model = Sequential()  
68  
69         model.add(Dense(7 * 7 * 64, input_dim=zdim))  
70         model.add(LeakyReLU(0.2))  
71         model.add(Reshape((7, 7, 64)))  
72  
73         model.add(Conv2D(128, (3, 3), padding="same"))  
74         model.add(LeakyReLU(0.2))  
75  
76         # 14x14 업샘플링  
77         model.add(Conv2DTranspose(128, (4, 4), strides=(2, 2), padding="same"))
```

(실습) MNIST 숫자 데이터를 GAN으로 생성하기

```
76     # 14x14 업샘플링
77     model.add(Conv2DTranspose(128, (4, 4), strides=(2, 2), padding="same"))
78     model.add(LeakyReLU(0.2))
79
80     # 28x28 업샘플링
81     model.add(Conv2DTranspose(128, (4, 4), strides=(2, 2), padding="same"))
82     model.add(LeakyReLU(0.2))
83
84     # 최종 출력 (tanh 사용)
85     model.add(Conv2D(1, (3, 3), padding="same", activation="tanh"))
86
87     return model
88
89
90     # -----
91     # 4. GAN 모델(G + D) 연결
92     # -----
93     def make_gan(G, D):
94         # GAN 학습 시에는 D의 가중치를 업데이트하지 않음
95         D.trainable = False
96
97         model = Sequential()
98         model.add(G)
99         model.add(D)
100
101         model.compile(
102             loss="binary_crossentropy",
```


(실습) MNIST 숫자 데이터를 GAN으로 생성하기

```
101     model.compile(  
102         loss="binary_crossentropy",  
103         optimizer=Adam(learning_rate=0.0002, beta_1=0.5)  
104     )  
105     return model  
106  
107  
108     # -----  
109     # 5. 진짜·가짜 샘플 생성 함수  
110     # -----  
111     def generate_real_samples(dataset, n_samples):  
112         # 진짜 이미지 선택  
113         idx = np.random.randint(0, dataset.shape[0], n_samples)  
114         x = dataset[idx]  
115         y = np.ones((n_samples, 1)) # 진짜는 1  
116         return x, y  
117  
118  
119     def generate_latent_points(zdim, n_samples):  
120         # 잠복 공간에서 랜덤 노이즈 샘플링  
121         return np.random.randn(n_samples, zdim)  
122  
123  
124     def generate_fake_samples(G, zdim, n_samples):  
125         # G(z)를 통해 가짜 이미지 생성  
126         z = generate_latent_points(zdim, n_samples)  
127         x = G.predict(z, verbose=0)  
128         y = np.zeros((n_samples, 1)) # 가짜는 0
```

(실습) MNIST 숫자 데이터를 GAN으로 생성하기

```
127     x = G.predict(z, verbose=0)
128     y = np.zeros((n_samples, 1)) # 가짜는 0
129     return x, y
130
131
132     # -----
133     # 6. GAN 학습 루프
134     # -----
135     def train(G, D, GAN, dataset, zdim, n_epochs=200, batch_size=128, verbose=1):
136         n_batch = dataset.shape[0] // batch_size
137
138         for epoch in range(n_epochs):
139             for _ in range(n_batch):
140
141                 # 분별망 학습(진짜 이미지)
142                 x_real, y_real = generate_real_samples(dataset, batch_size // 2)
143                 d_loss_real, _ = D.train_on_batch(x_real, y_real)
144
145                 # 분별망 학습(가짜 이미지)
146                 x_fake, y_fake = generate_fake_samples(G, zdim, batch_size // 2)
147                 d_loss_fake, _ = D.train_on_batch(x_fake, y_fake)
148
149                 # 생성망 학습(D를 속이도록 학습)
150                 z = generate_latent_points(zdim, batch_size)
151                 y_gan = np.ones((batch_size, 1)) # 생성망은 가짜를 진짜처럼 보이게 학습
152                 g_loss = GAN.train_on_batch(z, y_gan)
153
```

(실습) MNIST 숫자 데이터를 GAN으로 생성하기

```
153
154     if verbose:
155         print(f"Epoch {epoch+1}: D(real)={d_loss_real:.3f}, D(fake)={d_loss_fake:.3f}, G={g_loss:.3f}")
156
157     # 10 epoch마다 생성 결과 시각화
158     if (epoch + 1) % 10 == 0:
159         x_fake, y_fake = generate_fake_samples(G, zdim, 12)
160         plt.figure(figsize=(20, 2))
161         plt.suptitle(f"Epoch {epoch+1}")
162         for i in range(12):
163             plt.subplot(1, 12, i + 1)
164             plt.imshow((x_fake[i] + 1) / 2.0, cmap="gray")
165             plt.axis("off")
166         plt.show()
167
168
169     # -----
170     # 7. 모델 생성 및 학습 실행
171     # -----
172     D = make_discriminator((28, 28, 1))
173     G = make_generator(zdim)
174     GAN = make_gan(G, D)
175
176     train(G, D, GAN, x_train, zdim, verbose=1)
177
```